Softmax Regression

In softmax, we represent labels using one-hot encoding.

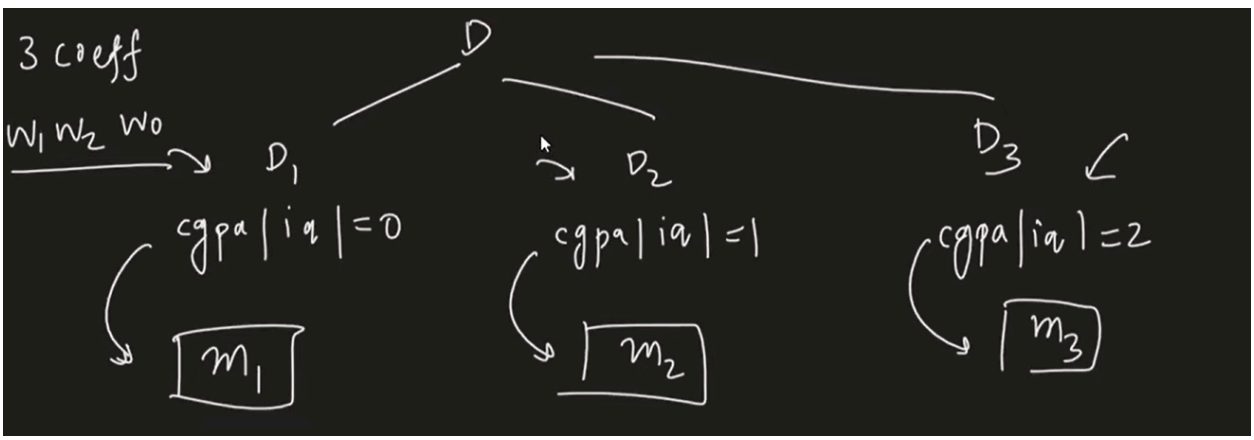
The model outputs a score for each class, and softmax converts these scores into probabilities that sum to 1.

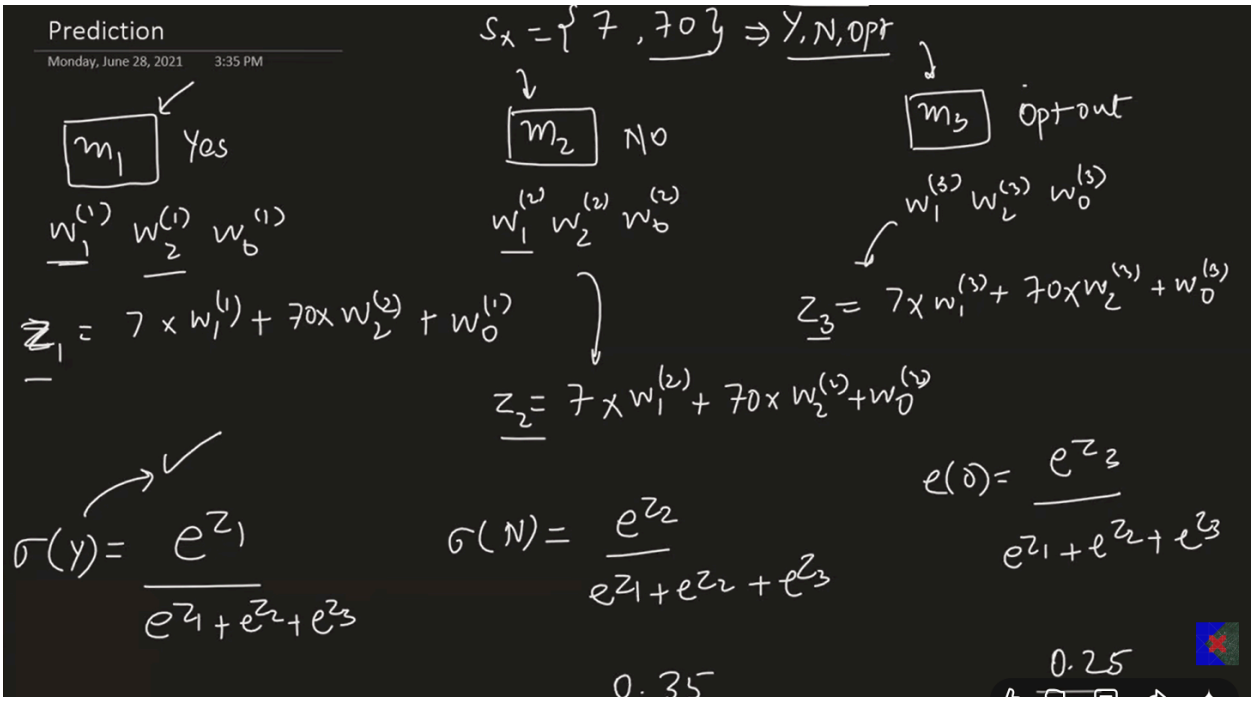
The class with the highest probability is chosen as the prediction.

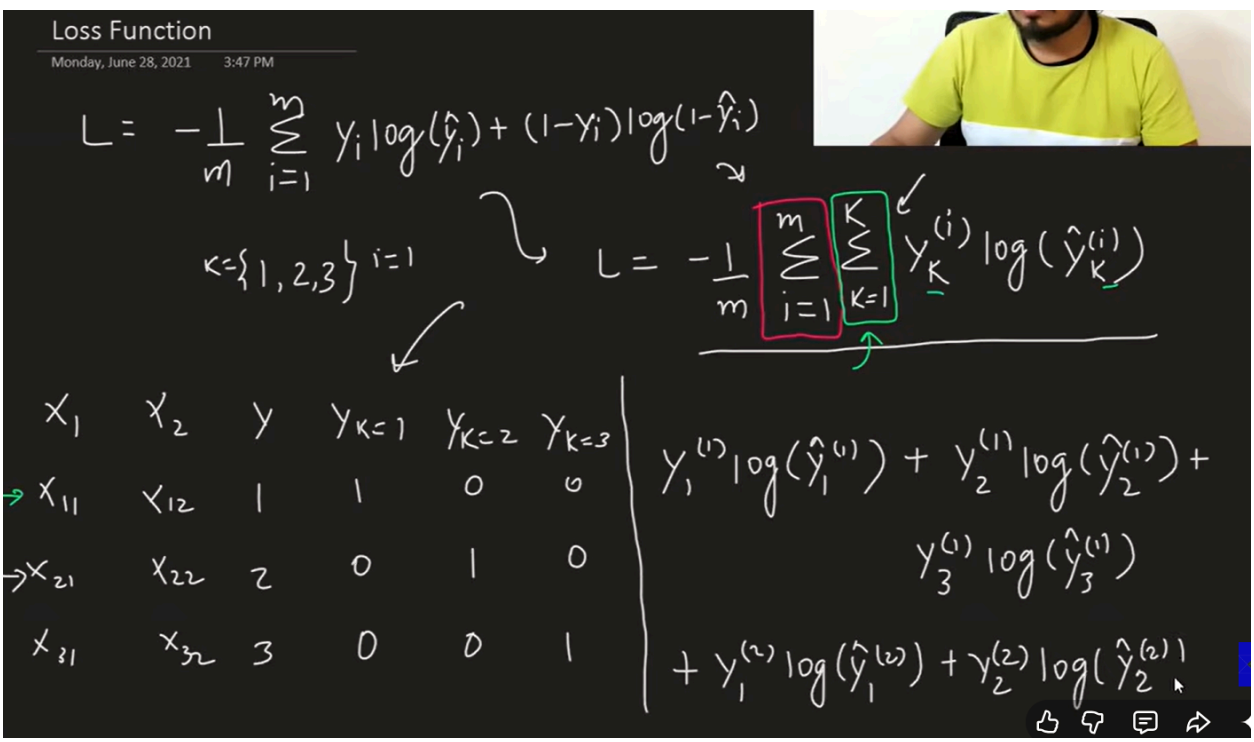
Unlike logistic regression, softmax does not apply sigmoid separately—it considers all classes together.

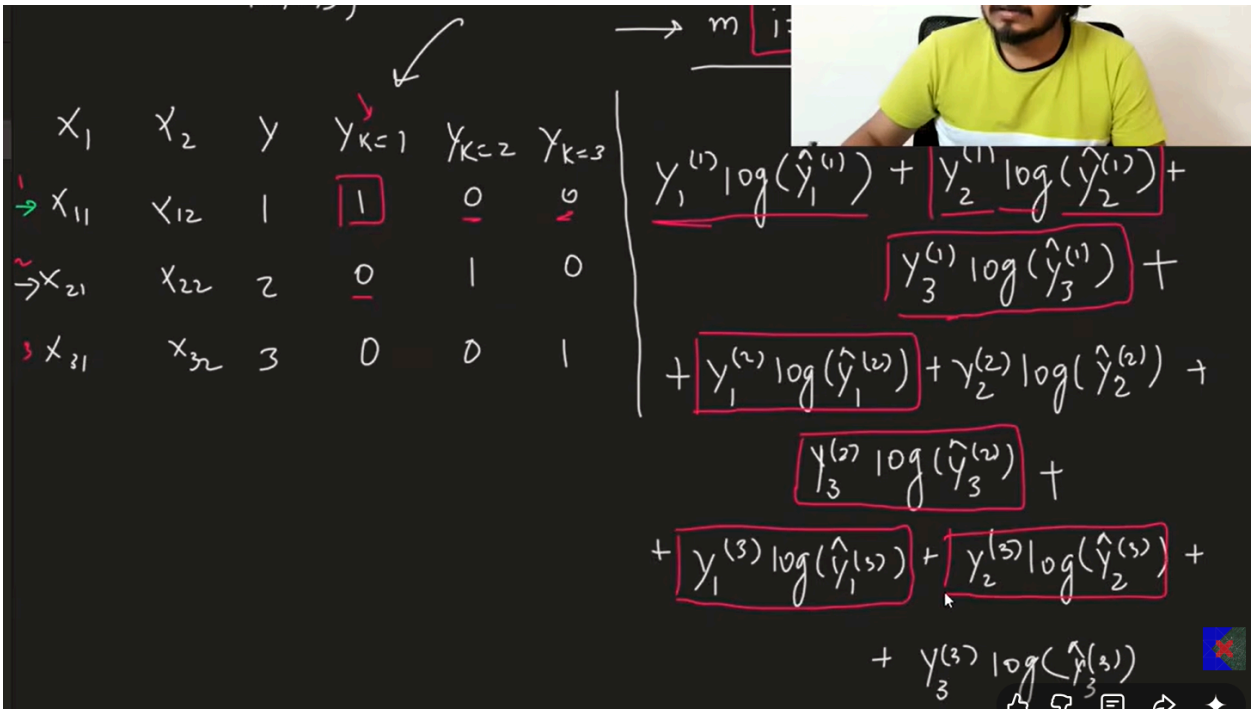
the following one is used in softmax

$$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$




Prediction
Monday, June 28, 2021 3:35 PM
$S_x = \{7, 70\} \Rightarrow Y, N, opt$
m_1 Yes
m_2 No
m_3 opt-out
$w_1^{(1)}\ w_2^{(1)}\ w_0^{(1)}$
$w_1^{(2)}\ w_2^{(2)}\ w_0^{(2)}$
$w_1^{(3)}\ w_2^{(3)}\ w_0^{(3)}$
$z_1 = 7 \times w_1^{(1)} + 70 \times w_2^{(2)} + w_0^{(1)}$
$z_2 = 7 \times w_1^{(2)} + 70 \times w_2^{(1)} + w_0^{(2)}$
$z_3 = 7 \times w_1^{(3)} + 70 \times w_2^{(3)} + w_0^{(3)}$
$\sigma(y) = \frac{e^{z_1}}{e^{z_1}+e^{z_2}+e^{z_3}}$
$\sigma(N) = \frac{e^{z_2}}{e^{z_1}+e^{z_2}+e^{z_3}}$
$e(o) = \frac{e^{z_3}}{e^{z_1}+e^{z_2}+e^{z_3}}$
0.35
0.25


Loss Function
Monday, June 28, 2021 3:47 PM
$L = -\frac{1}{m}\sum_{i=1}^{m} y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)$
$K = \{1, 2, 3\}$ $i=1$
$L = -\frac{1}{m}\sum_{i=1}^{m}\sum_{K=1}^{K} y_K^{(i)} \log(\hat{y}_K^{(i)})$
X_1 X_2 y $y_{K=1}$ $y_{K=2}$ $y_{K=3}$
X_{11} X_{12} 1 1 0 0
X_{21} X_{22} 2 0 1 0
X_{31} X_{32} 3 0 0 1
$y_1^{(1)} \log(\hat{y}_1^{(1)}) + y_2^{(1)} \log(\hat{y}_2^{(1)}) + y_3^{(1)} \log(\hat{y}_3^{(1)})$
$+ y_1^{(2)} \log(\hat{y}_1^{(2)}) + y_2^{(2)} \log(\hat{y}_2^{(2)})$


X_1 X_2 y $y_{K=1}$ $y_{K=2}$ $y_{K=3}$
X_{11} X_{12} 1 1 0 0
X_{21} X_{22} 2 0 1 0
X_{31} X_{32} 3 0 0 1
$y_1^{(1)} \log(\hat{y}_1^{(1)}) + y_2^{(1)} \log(\hat{y}_2^{(1)}) + y_3^{(1)} \log(\hat{y}_3^{(1)}) +$
$+ y_1^{(2)} \log(\hat{y}_1^{(2)}) + y_2^{(2)} \log(\hat{y}_2^{(2)}) + y_3^{(2)} \log(\hat{y}_3^{(2)}) +$
$+ y_1^{(3)} \log(\hat{y}_1^{(3)}) + y_2^{(3)} \log(\hat{y}_2^{(3)}) + y_3^{(3)} \log(\hat{y}_3^{(3)})$

$$L = \underline{y_1^{(1)}} \log(\hat{y}_1^{(1)}) + \underline{y_2^{(2)}} \log(\hat{y}_2^{(2)}) + \underline{y_3^{(3)}} \log(\hat{y}_3^{(3)})$$

$$\underline{\hat{y}_1^{(1)}}, \hat{y}_2^{(2)}, \hat{y}_3^{(3)}$$

softmax

$$\hat{y}_1^{(1)} = \sigma(w_1^{(1)} x_{11} + w_2^{(1)} x_{12} + w_0^{(1)})$$

$$y_2^{(2)} = \sigma(w_1^{(2)} x_{21} + w_2^{(2)} x_{22} + w_0^{(2)})$$

$$y_3^{(3)} = \sigma(w_1^{(3)} x_{31} + w_2^{(3)} x_{32} + w_0^{(3)})$$

Logistic regression
(2 possible output values)

$z = \vec{w} \cdot \vec{x} + b$

0.71

$a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1|\vec{x})$

$a_2 = 1 - a_1 = P(y=0|\vec{x})$

0.29

Softmax regression (4 possible outputs) $y = 1, 2, 3, 4$

$z_1 = \vec{w}_1 \cdot \vec{x} + b_1$

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=1|\vec{x})$$

$z_2 = \vec{w}_2 \cdot \vec{x} + b_2$

$$a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=2|\vec{x})$$

$z_3 = \vec{w}_3 \cdot \vec{x} + b_3$

$$a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=3|\vec{x})$$

$z_4 = \vec{w}_4 \cdot \vec{x} + b_4$

$$a_4 = \frac{e^{z_4}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=4|\vec{x})$$

More numerically accurate implementation of softmax

Softmax regression

$$(a_1, \dots, a_{10}) = g(z_1, \dots, z_{10})$$

$$\text{Loss} = L(\vec{a}, y) = \begin{cases} -\log a_1 & \text{if } y = 1 \\ \vdots & \\ -\log a_{10} & \text{if } y = 10 \end{cases}$$

```
model = Sequential([
  Dense(units=25, activation='relu'),
  Dense(units=15, activation='relu'),
  Dense(units=10, activation='softmax')
  ])
```

'linear'

~~model.compile(loss=SparseCategoricalCrossEntropy()~~)

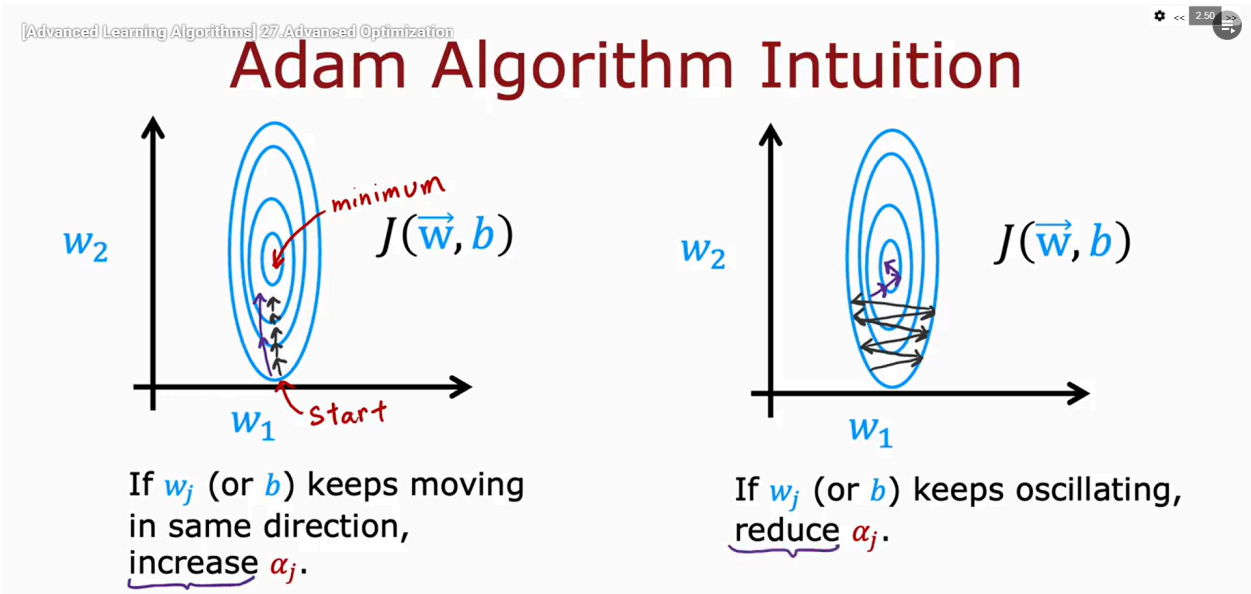
More Accurate

$$L(\vec{a}, y) = \begin{cases} -\log \frac{e^{z_1}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 1 \\ \vdots & \\ -\log \frac{e^{z_{10}}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 10 \end{cases}$$

```
model.compile(loss=SparseCategoricalCrossEntropy(from_logits=True))
```

this from_logits = true makes the loss calculation in accordance to z, not a where a is the sigmoid of z . That means it counts the most accurate loss before classification even occurs.

Adam Algorithm in gradient decent



MNIST Adam

model

```
model = Sequential([
        tf.keras.layers.Dense(units=25, activation='sigmoid'),
        tf.keras.layers.Dense(units=15, activation='sigmoid'),
        tf.keras.layers.Dense(units=10, activation='linear')
])
```

compile

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
  loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True))
```

fit

```
model.fit(X,Y,epochs=100)
```